

# KEMET

---

## *AI-Powered Egypt Tourism Assistant*

A Complete Data Engineering & AI Platform  
for Intelligent Travel Planning in Egypt

Graduation Project  
DEPI  
Track: Data Engineering

---

Kemet Development Team

April 16, 2026



# Contents

---

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Overview</b>                                                       | <b>3</b>  |
| 1.1      | Executive Summary . . . . .                                                   | 3         |
| 1.2      | Problem Statement . . . . .                                                   | 3         |
| 1.3      | Project Objectives . . . . .                                                  | 4         |
| 1.4      | Target Audience . . . . .                                                     | 4         |
| <b>2</b> | <b>Data Sources &amp; Collection Strategy</b>                                 | <b>5</b>  |
| 2.1      | Data Categories . . . . .                                                     | 5         |
| 2.1.1    | Tier 1 — Static Data (Scraped Once, Periodically Refreshed) . . . . .         | 5         |
| 2.1.2    | Tier 2 — Dynamic Data (Daily Automated Scraping via Apache Airflow) . . . . . | 6         |
| 2.1.3    | Tier 3 — User Generated Data . . . . .                                        | 6         |
| 2.2      | Web Scraping Strategy . . . . .                                               | 6         |
| <b>3</b> | <b>System Architecture</b>                                                    | <b>8</b>  |
| 3.1      | High-Level Architecture . . . . .                                             | 8         |
| 3.2      | Technology Stack . . . . .                                                    | 8         |
| 3.3      | API Integrations . . . . .                                                    | 9         |
| 3.3.1    | Google Maps Platform . . . . .                                                | 9         |
| 3.3.2    | OpenWeatherMap API . . . . .                                                  | 10        |
| <b>4</b> | <b>ETL Pipeline Design</b>                                                    | <b>11</b> |
| 4.1      | Pipeline Overview . . . . .                                                   | 11        |
| 4.1.1    | Extract Phase . . . . .                                                       | 11        |
| 4.1.2    | Transform Phase . . . . .                                                     | 11        |
| 4.1.3    | Load Phase . . . . .                                                          | 12        |
| 4.2      | Apache Airflow DAG Design . . . . .                                           | 12        |
| <b>5</b> | <b>Data Warehouse Design</b>                                                  | <b>14</b> |
| 5.1      | Star Schema . . . . .                                                         | 14        |
| 5.1.1    | Fact Tables . . . . .                                                         | 14        |
| 5.1.2    | Dimension Tables . . . . .                                                    | 14        |
| 5.1.3    | ERD Summary . . . . .                                                         | 15        |
| <b>6</b> | <b>AI Layer</b>                                                               | <b>16</b> |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| 6.1       | AI Chatbot (LangChain) . . . . .                    | 16        |
| 6.2       | Recommendation Engine . . . . .                     | 17        |
| 6.3       | Sentiment Analysis . . . . .                        | 17        |
| 6.4       | Smart Trip Optimizer . . . . .                      | 18        |
| <b>7</b>  | <b>Features Catalogue</b>                           | <b>19</b> |
| 7.1       | Core Tourism Features . . . . .                     | 19        |
| <b>8</b>  | <b>KPIs &amp; Success Metrics</b>                   | <b>21</b> |
| 8.1       | Business KPIs . . . . .                             | 21        |
| 8.2       | Data Engineering KPIs . . . . .                     | 21        |
| 8.3       | AI KPIs . . . . .                                   | 22        |
| <b>9</b>  | <b>System Design Diagrams</b>                       | <b>23</b> |
| 9.1       | Use Case Overview . . . . .                         | 23        |
| 9.2       | Data Flow Diagram (DFD Level 0 — Context) . . . . . | 24        |
| <b>10</b> | <b>UI/UX Design</b>                                 | <b>25</b> |
| 10.1      | Design Principles . . . . .                         | 25        |
| 10.2      | Key Screens . . . . .                               | 25        |
| 10.3      | Frontend Technology . . . . .                       | 25        |
| <b>11</b> | <b>Deployment &amp; DevOps</b>                      | <b>27</b> |
| 11.1      | Containerization with Docker . . . . .              | 27        |
| 11.2      | CI/CD with GitHub Actions . . . . .                 | 28        |
| <b>12</b> | <b>Testing &amp; Quality Assurance</b>              | <b>29</b> |
| 12.1      | Testing Strategy . . . . .                          | 29        |
| <b>13</b> | <b>Project Plan &amp; Milestones</b>                | <b>30</b> |
| 13.1      | Gantt Timeline (24 Weeks) . . . . .                 | 30        |
| 13.2      | Risk Assessment . . . . .                           | 30        |
| <b>14</b> | <b>Appendices</b>                                   | <b>32</b> |
| 14.1      | Flask REST API Endpoints . . . . .                  | 32        |
| 14.2      | Emergency Contacts (Sample Data) . . . . .          | 32        |
| 14.3      | Useful Egyptian Apps for Tourists . . . . .         | 32        |

# 1. Project Overview

---

## 1.1 Executive Summary

**Kemet** (named after the ancient Egyptian word for "*the Black Land*" — Egypt itself) is a comprehensive, AI-powered tourism intelligence platform designed to assist international and domestic tourists who wish to explore Egypt. The system integrates a full Data Engineering pipeline with an AI chatbot, recommendation engine, real-time pricing intelligence, safety analytics, and cultural guidance — all in one unified platform.

### Project Identity

|                       |                                              |
|-----------------------|----------------------------------------------|
| <b>Project Name</b>   | Kemet                                        |
| <b>Domain</b>         | Tourism Intelligence + Data Engineering + AI |
| <b>Track</b>          | Data Engineering — Ruwad Masr Al-Raqamia     |
| <b>Type</b>           | Graduation Project                           |
| <b>Primary Output</b> | AI Chatbot + Dashboard + ETL Pipeline        |

## 1.2 Problem Statement

International tourists visiting Egypt often face significant challenges:

- No single unified source for trip planning, pricing, and safety information.
- Vulnerability to price scams in souvenirs, transport, and accommodation.
- Difficulty navigating 125+ monuments, 35 archaeological sites, and 25 museums.
- Lack of real-time pricing for flights, hotels, and local products.
- Language barriers and limited knowledge of local culture, food, and customs.
- No AI-driven personalized trip planner tailored to budget and interests.

**Kemet** solves all of these by combining scraped data, APIs, and AI into one intelligent assistant.

## 1.3 Project Objectives

1. Build a complete ETL pipeline ingesting 30+ data sources with daily automation.
2. Design a Star Schema Data Warehouse storing all tourism data.
3. Develop an AI chatbot using LangChain for personalized trip planning.
4. Implement a recommendation engine based on budget, interests, and sentiment.
5. Create a real-time price intelligence system for flights, hotels, and products.
6. Integrate Google Maps API and Weather API for smart, context-aware recommendations.
7. Deliver a Streamlit/React dashboard and chat interface as the final UI.

## 1.4 Target Audience

- International tourists planning a visit to Egypt.
- Domestic tourists exploring new cities within Egypt.
- Travel agencies requiring data-driven destination insights.
- Egyptian tourism authorities monitoring KPIs and visitor patterns.

## 2. Data Sources & Collection Strategy

---

### 2.1 Data Categories

All data in Kemet is classified into three tiers based on update frequency:

#### 2.1.1 Tier 1 — Static Data (Scraped Once, Periodically Refreshed)

| Dataset                | Records   | Fields                                                                   |
|------------------------|-----------|--------------------------------------------------------------------------|
| Monuments              | 125       | name, location, description, open/close time, ticket price (EGP/Foreign) |
| Archaeological Sites   | 35        | name, location, description, hours, ticket prices                        |
| Museums                | 25        | name, location, description, hours, ticket prices                        |
| Historical Periods     | 24        | period name, time interval, description                                  |
| Monument Collections   | 160       | collection name, location (museum/city), description                     |
| Egyptian History       | Full      | Predynastic to Muhammad Ali Dynasty                                      |
| Tourism Types          | All       | Coastal, Archaeological, Medical, Religious, Adventure                   |
| Egyptian Culture       | Extensive | food, crafts, clothes, handmade products                                 |
| Egyptian Words Guide   | 200+      | common phrases, pronunciation, context                                   |
| Egyptian Sports Events | 50+       | matches, venues, booking links                                           |
| Celebrity Visits       | 30+       | names, Instagram proof posts, visited locations                          |

Table 2.1: Static Data Sources

**Source:** <https://egymonuments.gov.eg/en/> (monuments, sites, museums, collections)

### 2.1.2 Tier 2 — Dynamic Data (Daily Automated Scraping via Apache Airflow)

| Dataset         | Records  | Fields                                                                                                   |
|-----------------|----------|----------------------------------------------------------------------------------------------------------|
| Top Restaurants | 20/city  | name, city (Alex, Cairo, Hurghada, Sharm, Luxor, Aswan, Alamein, Al Gouna), rating, cuisine, price range |
| Hotels          | 51+      | name, rating, price/night, address, booking link, room info                                              |
| Airbnb Listings | Varied   | location, price, amenities, host rating                                                                  |
| Flight Prices   | Daily    | origin, destination, date, airline, price (USD/EGP)                                                      |
| Product Prices  | Daily    | category (food/electronics), price for foreigners vs locals                                              |
| Souvenir Prices | 100+     | name, type, fair price (anti-scam reference)                                                             |
| Safety Rates    | Per city | safety index, incident reports, tourist advisories                                                       |
| Events          | Weekly   | sports matches, festivals, concerts, booking info                                                        |

Table 2.2: Dynamic Data Sources (Daily Pipeline)

### 2.1.3 Tier 3 — User Generated Data

- Tourist reviews and ratings per place.
- Uploaded trip photos and experiences.
- Social media shares and adventure stories.
- Chatbot conversation history (anonymized).

## 2.2 Web Scraping Strategy

```

1 # Tools: BeautifulSoup + Selenium + Scrapy
2 # Scheduler: Apache Airflow DAGs
3
4 from bs4 import BeautifulSoup
5 import requests
6 import pandas as pd
7 from datetime import datetime
8
9 class KemetScraper:
10     def __init__(self, source_url: str, dataset_name: str):
11         self.source_url = source_url
12         self.dataset_name = dataset_name
13         self.timestamp = datetime.utcnow()
14
15     def scrape_monuments(self) -> pd.DataFrame:
16         """Scrape 125 monuments from egymonuments.gov.eg"""

```

```
17     response = requests.get(self.source_url)
18     soup = BeautifulSoup(response.text, 'html.parser')
19     records = []
20     for item in soup.select('.monument-card'):
21         records.append({
22             'name': item.select_one('.title').text.strip(),
23             'location': item.select_one('.location').text.strip
24         },
25             'description': item.select_one('.desc').text.strip(),
26             'open_time': item.select_one('.open').text.strip(),
27             'close_time': item.select_one('.close').text.strip(),
28             'ticket_egp': item.select_one('.price-egp').text.
29             strip(),
30             'ticket_foreign': item.select_one('.price-foreign').text.
31             strip(),
32             'scraped_at': self.timestamp
33         })
34     return pd.DataFrame(records)
35
36     def scrape_flights(self, origin: str, destination: str) -> pd.
37     DataFrame:
38         """Daily flight price scraping"""
39         # Uses Selenium for JS-rendered pages
40         pass
41
42     def scrape_hotels(self) -> pd.DataFrame:
43         """Hotel prices with daily updates"""
44         pass
```

Listing 2.1: Scraping Pipeline — Core Logic

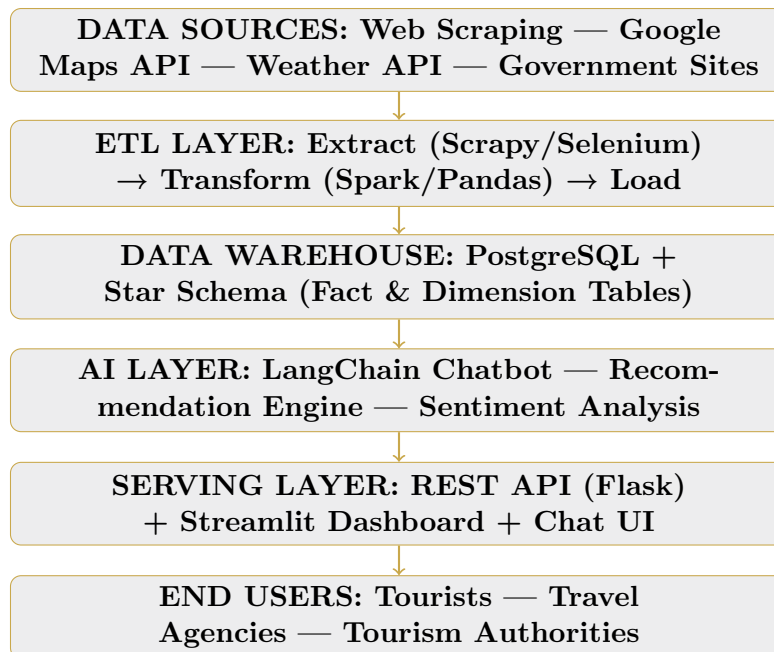


## 3. System Architecture

---

### 3.1 High-Level Architecture

The Kemet platform follows a layered architecture:



### 3.2 Technology Stack

| Layer             | Technology                              | Purpose                               |
|-------------------|-----------------------------------------|---------------------------------------|
| Data Collection   | Python, Scrapy, Selenium, BeautifulSoup | Web scraping static & dynamic data    |
| Orchestration     | Apache Airflow                          | Daily DAG scheduling for all scrapers |
| Stream Processing | Apache Kafka                            | Real-time price event streaming       |
| Batch Processing  | Apache Spark                            | Large-scale data transformation       |
| Storage (Raw)     | Azure Blob Storage / AWS S3             | Data Lake for raw scraped files       |

| Layer            | Technology                 | Purpose                                  |
|------------------|----------------------------|------------------------------------------|
| Data Warehouse   | PostgreSQL / Azure Synapse | Star Schema, analytics queries           |
| Data Factory     | Azure Data Factory         | Cloud ETL pipelines                      |
| Containerization | Docker                     | Isolated, reproducible environments      |
| Big Data         | Hadoop (HDFS)              | Distributed file storage if needed       |
| Backend API      | Flask (Python)             | REST endpoints for frontend & chatbot    |
| AI / NLP         | LangChain + OpenAI/Gemini  | Conversational AI, trip planning         |
| ML Models        | Scikit-learn, HuggingFace  | Recommendations, sentiment analysis      |
| Frontend         | Streamlit / React          | Dashboard + Chat Interface               |
| Maps             | Google Maps Platform API   | Location display, routing, nearby search |
| Weather          | OpenWeatherMap API         | Real-time weather for trip planning      |
| CI/CD            | GitHub Actions             | Automated build, test, deploy            |
| Version Control  | Git / GitHub               | Source code management                   |

Table 3.1: Full Technology Stack

## 3.3 API Integrations

### 3.3.1 Google Maps Platform

```

1 import googlemaps
2 from datetime import datetime
3
4 gmaps = googlemaps.Client(key='YOUR_API_KEY')
5
6 def get_nearby_places(lat: float, lng: float, place_type: str):
7     """Find restaurants/ATMs/hospitals near tourist location"""
8     results = gmaps.places_nearby(
9         location=(lat, lng),
10        radius=2000,
11        type=place_type # 'restaurant' | 'hospital' | 'atm'
12    )
13    return results['results']
14
15 def get_directions(origin: str, destination: str, mode='transit'):
16     """Optimized route from hotel to monument"""
17     directions = gmaps.directions(
18         origin=origin,
19         destination=destination,
20         mode=mode,
21         departure_time=datetime.now()
22     )
23    return directions

```

Listing 3.1: Google Maps API — Nearby Places

### 3.3.2 OpenWeatherMap API

```
1 import requests
2
3 def get_weather(city: str) -> dict:
4     """Fetch current + 7-day forecast for trip planning"""
5     url = f"https://api.openweathermap.org/data/2.5/forecast"
6     params = {
7         'q': city,
8         'appid': 'YOUR_API_KEY',
9         'units': 'metric',
10        'cnt': 7
11    }
12    response = requests.get(url, params=params)
13    data = response.json()
14    return {
15        'city': city,
16        'temp': data['list'][0]['main']['temp'],
17        'description': data['list'][0]['weather'][0]['description'],
18        'forecast': data['list']
19    }
20
21 def recommend_based_on_weather(city: str) -> str:
22     """AI suggestion based on weather"""
23     weather = get_weather(city)
24     if weather['temp'] > 38:
25         return "High heat advisory: recommend indoor museums or early morning visits."
26     elif 'rain' in weather['description']:
27         return "Rain expected: recommend indoor attractions and covered bazaars."
28     else:
29         return "Perfect weather for outdoor monuments and beach activities!"
```

Listing 3.2: Weather-Aware Trip Planning

## 4. ETL Pipeline Design

---

### 4.1 Pipeline Overview

The Kemet ETL pipeline is the core data engine. It runs on a combination of batch (Airflow DAGs) and near-real-time (Kafka) processing.

#### 4.1.1 Extract Phase

- **Static scraping:** Government monument/museum pages (run once + monthly refresh)
- **Dynamic scraping:** Flights, hotels, restaurants, prices (daily via Airflow)
- **API pulls:** Google Maps, OpenWeatherMap (on-demand + cached)
- **User inputs:** Reviews, ratings, uploaded media

#### 4.1.2 Transform Phase

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, regexp_replace, to_date, trim,
   udf
3 from pyspark.sql.types import FloatType
4
5 spark = SparkSession.builder.appName("KemetETL").getOrCreate()
6
7 def transform_hotels(raw_df):
8     """Clean and enrich hotel data"""
9     return (raw_df
10            .dropDuplicates(['Place_Name', 'Address'])
11            .filter(col('Price').isNotNull())
12            .withColumn('Price_Clean',
13                       regexp_replace(col('Price'), '[^0-9.]', '' ).cast(FloatType())
14            ))
15
16            .withColumn('Price_USD', col('Price_Clean') / 50.0) # EGP to
   USD
17            .withColumn('Rating_Numeric', col('Rating').cast(FloatType()))
18            .select('Place_Name', 'Rating_Numeric', 'Price_Clean',
19                   'Price_USD', 'Address', 'Link', 'Room_Info')
```

```

20 def compute_popularity_score(reviews_df, visits_df):
21     """Feature engineering: popularity index per location"""
22     return (reviews_df
23             .join(visits_df, on='place_id', how='left')
24             .withColumn('popularity_score',
25                         (col('avg_rating') * 0.4) +
26                         (col('review_count') * 0.3) +
27                         (col('visit_count') * 0.3))
28             )

```

Listing 4.1: Data Transformation with PySpark

### 4.1.3 Load Phase

Data is loaded into the PostgreSQL Data Warehouse following the Star Schema described in Chapter 5. A summary of loading strategy:

- **Upsert (MERGE)** for dynamic tables (flights, hotels, prices).
- **Insert-only** for fact tables (reviews, bookings, visits).
- **Truncate + reload** for slowly changing dimension tables.

## 4.2 Apache Airflow DAG Design

```

1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime, timedelta
4
5 default_args = {
6     'owner': 'kemet',
7     'depends_on_past': False,
8     'start_date': datetime(2025, 1, 1),
9     'retries': 2,
10    'retry_delay': timedelta(minutes=5),
11    'email_on_failure': True,
12 }
13
14 with DAG(
15     dag_id='kemet_daily_prices',
16     default_args=default_args,
17     schedule_interval='0 2 * * *', # Every day at 02:00 UTC
18     catchup=False,
19     tags=['kemet', 'prices', 'daily'],
20 ) as dag:
21
22     scrape_flights = PythonOperator(
23         task_id='scrape_flight_prices',
24         python_callable=scrape_flights_func,
25     )
26     scrape_hotels = PythonOperator(
27         task_id='scrape_hotel_prices',
28         python_callable=scrape_hotels_func,
29     )
30     transform_data = PythonOperator(
31         task_id='transform_and_clean',

```

```
32     python_callable=transform_prices_func,
33 )
34 load_dwh = PythonOperator(
35     task_id='load_to_warehouse',
36     python_callable=load_to_postgres_func,
37 )
38
39 [scrape_flights, scrape_hotels] >> transform_data >> load_dwh
```

Listing 4.2: Airflow DAG — Daily Price Pipeline

## 5. Data Warehouse Design

---

### 5.1 Star Schema

The Kemet Data Warehouse uses a Star Schema optimized for analytical queries. The schema consists of 4 Fact tables and 5 Dimension tables.

#### 5.1.1 Fact Tables

| Table         | Description & Key Measures                                                                        |
|---------------|---------------------------------------------------------------------------------------------------|
| Fact_Reviews  | Tourist reviews per place. Measures: rating, sentiment_score, review_count                        |
| Fact_Prices   | Daily price records for flights, hotels, products. Measures: price_egp, price_usd, price_change_% |
| Fact_Bookings | Hotel and tour bookings. Measures: booking_value, nights, occupancy_rate                          |
| Fact_Visits   | Place visit logs. Measures: visitor_count, avg_time_spent, peak_hour                              |

Table 5.1: Fact Tables

#### 5.1.2 Dimension Tables

| Table        | Key Attributes                                                 | Connected Facts         |
|--------------|----------------------------------------------------------------|-------------------------|
| Dim_Location | city, governorate, latitude, longitude, region_type            | All Fact tables         |
| Dim_Time     | date, day_of_week, month, season, is_holiday                   | All Fact tables         |
| Dim_Place    | place_id, name, category (restaurant/monument/hotel), capacity | Reviews, Visits, Prices |
| Dim_User     | user_id, nationality, budget_range, travel_style               | Reviews, Bookings       |

| Table        | Key Attributes                                   | Connected Facts |
|--------------|--------------------------------------------------|-----------------|
| Dim_Category | category_id, type (food/culture/adventure), tags | Reviews, Visits |

Table 5.2: Dimension Tables

### 5.1.3 ERD Summary

```

1  -- Dimension: Location
2  CREATE TABLE Dim_Location (
3      location_id    SERIAL PRIMARY KEY,
4      city           VARCHAR(100),
5      governorate    VARCHAR(100),
6      latitude       DECIMAL(9,6),
7      longitude      DECIMAL(9,6),
8      region_type    VARCHAR(50) -- coastal | urban | historical | desert
9  );
10
11 -- Dimension: Place
12 CREATE TABLE Dim_Place (
13     place_id        SERIAL PRIMARY KEY,
14     name            VARCHAR(255),
15     category        VARCHAR(100),
16     location_id     INT REFERENCES Dim_Location(location_id),
17     capacity        INT,
18     ticket_egg      DECIMAL(10,2),
19     ticket_foreign  DECIMAL(10,2),
20     google_rating   DECIMAL(3,2),
21     opening_time    TIME,
22     closing_time    TIME
23 );
24
25 -- Fact: Reviews
26 CREATE TABLE Fact_Reviews (
27     review_id       SERIAL PRIMARY KEY,
28     place_id        INT REFERENCES Dim_Place(place_id),
29     user_id         INT REFERENCES Dim_User(user_id),
30     date_id         INT REFERENCES Dim_Time(date_id),
31     rating          DECIMAL(3,2),
32     sentiment_score DECIMAL(5,4),
33     sentiment_label VARCHAR(20),
34     review_text     TEXT
35 );
36
37 -- Fact: Prices (Daily Snapshot)
38 CREATE TABLE Fact_Prices (
39     price_id        SERIAL PRIMARY KEY,
40     place_id        INT REFERENCES Dim_Place(place_id),
41     date_id         INT REFERENCES Dim_Time(date_id),
42     price_egg       DECIMAL(12,2),
43     price_usd       DECIMAL(10,2),
44     price_source    VARCHAR(100),
45     category        VARCHAR(50) -- flight | hotel | product | souvenir
46 );

```

Listing 5.1: Core Schema DDL



## 6. AI Layer

---

### 6.1 AI Chatbot (LangChain)

The Kemet AI Chatbot is built using LangChain with a Retrieval-Augmented Generation (RAG) approach, connecting to the PostgreSQL warehouse as the knowledge base.

```
1 from langchain_community.chat_models import ChatOpenAI
2 from langchain.chains import ConversationalRetrievalChain
3 from langchain_community.vectorstores import FAISS
4 from langchain_community.embeddings import OpenAIEmbeddings
5 from langchain.memory import ConversationBufferMemory
6 from langchain.prompts import PromptTemplate
7
8 KEMET_SYSTEM_PROMPT = """
9 You are Kemet, an expert AI guide for tourism in Egypt.
10 You have deep knowledge of:
11 - 125 monuments, 35 archaeological sites, 25 museums
12 - Best restaurants in Alex, Cairo, Hurghada, Sharm, Luxor, Aswan
13 - Real-time hotel and flight prices
14 - Safety ratings per city
15 - Egyptian culture, food, history, and crafts
16 - Emergency contacts and travel apps
17
18 When planning a trip, always consider:
19 1. User's budget (ask if not provided)
20 2. Travel dates (for weather and events)
21 3. Interests (history / beach / food / adventure)
22 4. Safety recommendations
23
24 Respond in the user's language. Be friendly, helpful, and concise.
25 """
26
27 def build_kemet_chain(vector_store):
28     llm = ChatOpenAI(model="gpt-4", temperature=0.3)
29     memory = ConversationBufferMemory(
30         memory_key="chat_history",
31         return_messages=True
32     )
33     chain = ConversationalRetrievalChain.from_llm(
34         llm=llm,
35         retriever=vector_store.as_retriever(search_kwargs={"k": 5}),
36         memory=memory,
37         verbose=False
```

```

38     )
39     return chain
40
41 def plan_trip(user_query: str, chain) -> str:
42     """Main entry point for trip planning queries"""
43     response = chain({"question": user_query})
44     return response['answer']

```

Listing 6.1: LangChain RAG Chatbot — Core Setup

## 6.2 Recommendation Engine

```

1  import pandas as pd
2  from sklearn.feature_extraction.text import TfidfVectorizer
3  from sklearn.metrics.pairwise import cosine_similarity
4
5  class KemetRecommender:
6      def __init__(self, places_df: pd.DataFrame):
7          self.df = places_df
8          self.tfidf = TfidfVectorizer(stop_words='english')
9          self._build_index()
10
11     def _build_index(self):
12         """Build TF-IDF similarity matrix on place descriptions"""
13         corpus = self.df['description'] + ' ' + self.df['category']
14         self.tfidf_matrix = self.tfidf.fit_transform(corpus)
15         self.sim_matrix = cosine_similarity(self.tfidf_matrix)
16
17     def recommend(self, interests: list, budget: float,
18                  city: str = None, top_n: int = 10) -> pd.DataFrame:
19         """Budget + interest-based recommendations"""
20         query = ' '.join(interests)
21         query_vec = self.tfidf.transform([query])
22         scores = cosine_similarity(query_vec, self.tfidf_matrix).flatten
23         ()
24         self.df['relevance_score'] = scores
25
26         filtered = self.df[self.df['ticket_foreign'] <= budget]
27         if city:
28             filtered = filtered[filtered['city'] == city]
29
30         return (filtered
31                 .sort_values('relevance_score', ascending=False)
32                 .head(top_n)[['name', 'city', 'category',
33                              'ticket_foreign', 'relevance_score']])

```

Listing 6.2: Content-Based Recommendation System

## 6.3 Sentiment Analysis

```

1  from transformers import pipeline
2  import pandas as pd
3
4  class KemetSentimentAnalyzer:
5      def __init__(self):

```

```

6         self.model = pipeline(
7             "sentiment-analysis",
8             model="cardiffnlp/twitter-roberta-base-sentiment-latest"
9         )
10
11     def analyze_reviews(self, reviews_df: pd.DataFrame) -> pd.DataFrame:
12         """Batch sentiment analysis on tourist reviews"""
13         texts = reviews_df['review_text'].tolist()
14         results = self.model(texts, batch_size=32, truncation=True)
15         reviews_df['sentiment_label'] = [r['label'] for r in results]
16         reviews_df['sentiment_score'] = [r['score'] for r in results]
17         return reviews_df
18
19     def get_place_sentiment_kpi(self, place_name: str,
20                                df: pd.DataFrame) -> dict:
21         place_reviews = df[df['place_name'] == place_name]
22         pos = (place_reviews['sentiment_label'] == 'positive').mean()
23         return {
24             'place': place_name,
25             'positive_rate': round(pos * 100, 2),
26             'review_count': len(place_reviews),
27             'avg_score': round(place_reviews['sentiment_score'].mean(),
28
4)         }

```

Listing 6.3: Sentiment Analysis Pipeline

## 6.4 Smart Trip Optimizer

The Smart Trip Optimizer combines Maps API distances, weather data, place ratings, and user preferences to generate an optimized daily itinerary:

1. Fetch user preferences: city, budget, interests, trip duration.
2. Query recommendation engine for top candidate places.
3. Retrieve weather forecast for trip dates.
4. Calculate pairwise distances via Google Maps Distance Matrix API.
5. Apply Nearest Neighbor heuristic to minimize daily travel time.
6. Inject weather warnings and opening hour constraints.
7. Return a structured day-by-day itinerary with times and costs.

## 7. Features Catalogue

---

### 7.1 Core Tourism Features

| Feature             | Description                                                                           | Data Tier |
|---------------------|---------------------------------------------------------------------------------------|-----------|
| Trip Planner        | AI generates complete day-by-day itinerary based on city, duration, budget, interests | AI        |
| Hotel Finder        | Budget-based hotel suggestions with booking links                                     | Dynamic   |
| Flight Tracker      | Daily cheapest flights to Egyptian destinations                                       | Dynamic   |
| Restaurant Map      | Top 20 restaurants per city with Google Maps pins                                     | Dynamic   |
| Monument Explorer   | 125 monuments with ticket prices, hours, descriptions                                 | Static    |
| Museum Guide        | 25 museums with full cultural context                                                 | Static    |
| Safety Dashboard    | Per-city safety index, tourist advisories, safe vs unsafe zones                       | Dynamic   |
| Weather Planner     | 7-day forecast integrated into trip suggestions                                       | API       |
| Anti-Scam Prices    | Souvenir and product fair-price reference guide                                       | Static    |
| Sentiment Rankings  | AI-analyzed review scores per attraction                                              | AI        |
| Egyptian Food Guide | Must-try dishes linked to serving restaurants                                         | Static    |
| Cultural Guide      | Crafts, clothes, customs, Arabic phrases                                              | Static    |
| Emergency Contacts  | Police, ambulance, hospital numbers per city                                          | Static    |
| Sports Events       | Football fixtures (Ahly vs Zamalek), venue maps, booking                              | Dynamic   |
| Airbnb Finder       | Alternative accommodation with prices                                                 | Dynamic   |
| Internal Transport  | Trains, buses, internal flights, prices, booking                                      | Dynamic   |
| SIM / eSIM Guide    | Data packages, providers, how to top up                                               | Static    |
| Delivery Apps Guide | Talabat, Instashop, Vezeeta, pharmacy apps                                            | Static    |
| Celebrity Visits    | Famous visitors with Instagram proof posts                                            | Static    |
| Photo Spots Guide   | Best photography spots and composition tips per location                              | Static    |
| Historical Periods  | 24 periods from Predynastic to Muhammad Ali Dynasty                                   | Static    |

---

| Feature | Description | Data Tier |
|---------|-------------|-----------|
|---------|-------------|-----------|

---

Table 7.1: Complete Feature Catalogue

## 8. KPIs & Success Metrics

---

### 8.1 Business KPIs

| KPI                     | Definition                                              | Target                  |
|-------------------------|---------------------------------------------------------|-------------------------|
| User Satisfaction Rate  | % positive chatbot ratings                              | $\geq 85\%$             |
| Booking Conversion Rate | Users who booked after recommendation / total users     | $\geq 15\%$             |
| Trip Cost Optimization  | Avg saving vs non-optimized trip (USD)                  | $\geq \$50/\text{trip}$ |
| Popularity Index        | Weighted score per location (rating + visits + reviews) | Computed                |
| Safety Score            | Per-city index (0–100)                                  | Monitored               |
| Price Scam Prevention   | Users who used fair-price guide before purchase         | Tracked                 |

Table 8.1: Business KPIs

### 8.2 Data Engineering KPIs

| KPI                   | Definition                                   | Target          |
|-----------------------|----------------------------------------------|-----------------|
| Data Freshness        | Age of newest price record in DWH            | $\leq 24$ hours |
| Pipeline Success Rate | % of daily DAG runs completing without error | $\geq 99\%$     |
| Scraping Coverage     | % of target sources successfully scraped     | $\geq 95\%$     |
| ETL Latency           | Time from scrape trigger to DWH availability | $\leq 30$ min   |
| Data Quality Score    | % of records passing all validation checks   | $\geq 98\%$     |

Table 8.2: Data Engineering KPIs

## 8.3 AI KPIs

| KPI                     |       | Definition                                   | Target           |
|-------------------------|-------|----------------------------------------------|------------------|
| Recommendation Accuracy | Accu- | Precision@10 for place recommendations       | $\geq 80\%$      |
| Chatbot Response Time   |       | End-to-end latency per query                 | $\leq 3$ seconds |
| Sentiment Accuracy      |       | F1 score on labeled review test set          | $\geq 88\%$      |
| Trip Acceptance Rate    |       | % of generated itineraries accepted by user  | $\geq 70\%$      |
| Weather Accuracy        |       | Forecast precision for smart recommendations | API-dependent    |

Table 8.3: AI KPIs

## 9. System Design Diagrams

---

### 9.1 Use Case Overview

#### Primary Actors:

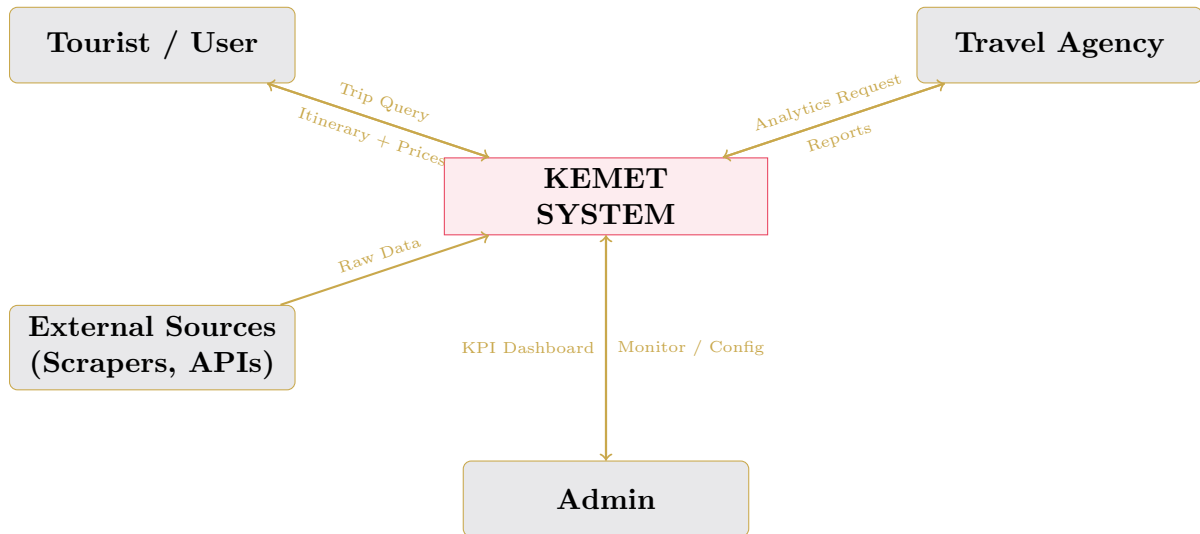
- **Tourist** — Plans trips, searches places, reads reviews, books hotels.
- **Travel Agency** — Accesses bulk recommendations and analytics.
- **Admin** — Manages data sources, monitors pipelines and KPIs.
- **AI System** — Generates recommendations, analyses sentiment, plans routes.

#### Core Use Cases:

1. UC-01: Plan a Trip (Tourist → AI Chatbot → DWH)
2. UC-02: Search Places by City/Budget/Interest
3. UC-03: Get Real-Time Flight Prices
4. UC-04: View Safety Map for a City
5. UC-05: Check Fair Price Before Buying Souvenir
6. UC-06: Read Sentiment-Ranked Reviews
7. UC-07: Generate Weather-Aware Day Plan
8. UC-08: Find Nearby Restaurants/ATMs/Hospitals
9. UC-09: Share Travel Experience
10. UC-10: Monitor Tourism KPIs (Admin Dashboard)



## 9.2 Data Flow Diagram (DFD Level 0 — Context)



## 10. UI/UX Design

---

### 10.1 Design Principles

- **Color Palette:** Gold (#C9A84C), Dark Navy (#1A1A2E), White (#FFFFFF) — inspired by ancient Egyptian aesthetics.
- **Typography:** Clean sans-serif (Inter/Poppins) for readability.
- **Language:** English primary, Arabic secondary.
- **Accessibility:** High contrast ratios, large touch targets for mobile.
- **Mobile-First:** Responsive design for tourists using smartphones.

### 10.2 Key Screens

1. **Home / Landing Page** — Hero section with Egypt imagery, quick search bar (city + dates + budget).
2. **AI Chat Interface** — WhatsApp-style chat with Kemet AI, quick-reply buttons.
3. **Trip Timeline View** — Horizontal day-by-day itinerary with map pins.
4. **Explore Map** — Google Maps embedded with colored pins per category.
5. **Price Intelligence Dashboard** — Line charts for flight/hotel price trends.
6. **Place Detail Page** — Photos, reviews, sentiment score, ticket info, weather widget.
7. **Safety Map** — Heatmap of safety index per city/district.
8. **Admin Dashboard** — Pipeline health, KPI gauges, data freshness indicators.

### 10.3 Frontend Technology

```
1 import streamlit as st
2
3 st.set_page_config(
4     page_title="Kemet      Egypt AI Guide",
5     page_icon="      ",
```

```
6     layout="wide"
7 )
8
9 # Sidebar navigation
10 with st.sidebar:
11     st.image("kemet_logo.png", width=150)
12     page = st.radio("Navigate", [
13         "Home", "Chat with Kemet",
14         "Explore Map", "Prices",
15         "Safety", "Analytics"
16     ])
17
18 # Chat page
19 if page == "Chat with Kemet":
20     st.title("Chat with Kemet AI")
21     if "messages" not in st.session_state:
22         st.session_state.messages = []
23
24     for msg in st.session_state.messages:
25         with st.chat_message(msg["role"]):
26             st.write(msg["content"])
27
28     user_input = st.chat_input("Ask me anything about Egypt...")
29     if user_input:
30         st.session_state.messages.append(
31             {"role": "user", "content": user_input}
32         )
33         response = kemet_chain(user_input)
34         st.session_state.messages.append(
35             {"role": "assistant", "content": response}
36         )
37     st.rerun()
```

Listing 10.1: Streamlit Chat Interface — Core

# 11. Deployment & DevOps

---

## 11.1 Containerization with Docker

```
1 version: '3.9'
2 services:
3   postgres:
4     image: postgres:15
5     environment:
6       POSTGRES_DB: kemet_dwh
7       POSTGRES_USER: kemet
8       POSTGRES_PASSWORD: ${DB_PASSWORD}
9     volumes:
10      - pgdata:/var/lib/postgresql/data
11     ports:
12      - "5432:5432"
13
14   airflow:
15     image: apache/airflow:2.8.0
16     depends_on: [postgres]
17     environment:
18       AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://kemet:${DB_PASSWORD}@postgres/kemet_dwh
19     volumes:
20      - ./dags:/opt/airflow/dags
21     ports:
22      - "8080:8080"
23
24   backend:
25     build: ./backend
26     depends_on: [postgres]
27     environment:
28       DATABASE_URL: postgresql://kemet:${DB_PASSWORD}@postgres/kemet_dwh
29       OPENAI_API_KEY: ${OPENAI_API_KEY}
30       GOOGLE_MAPS_KEY: ${GOOGLE_MAPS_KEY}
31     ports:
32      - "5000:5000"
33
34   frontend:
35     build: ./frontend
36     depends_on: [backend]
37     ports:
38      - "8501:8501"
39
```

```
40 volumes:
41   pgdata:
```

Listing 11.1: docker-compose.yml — Kemet Services

## 11.2 CI/CD with GitHub Actions

```
1 name: Kemet CI/CD Pipeline
2
3 on:
4   push:
5     branches: [main, develop]
6
7 jobs:
8   test:
9     runs-on: ubuntu-latest
10    steps:
11      - uses: actions/checkout@v4
12      - name: Set up Python
13        uses: actions/setup-python@v5
14        with: { python-version: '3.11' }
15      - run: pip install -r requirements.txt
16      - run: pytest tests/ -v --tb=short
17
18 build-and-deploy:
19   needs: test
20   runs-on: ubuntu-latest
21   if: github.ref == 'refs/heads/main'
22   steps:
23     - uses: actions/checkout@v4
24     - name: Build Docker images
25       run: docker-compose build
26     - name: Deploy to Azure
27       run: |
28         az acr login --name kemetregistry
29         docker-compose push
```

Listing 11.2: .github/workflows/deploy.yml

## 12. Testing & Quality Assurance

### 12.1 Testing Strategy

| Test Type          | Scope                               | Tool                    |
|--------------------|-------------------------------------|-------------------------|
| Unit Tests         | Scrapers, transformers, recommender | pytest                  |
| Integration Tests  | ETL pipeline end-to-end             | pytest + testcontainers |
| API Tests          | Flask REST endpoints                | pytest + requests       |
| Load Tests         | Chatbot under concurrent users      | Locust                  |
| Data Quality Tests | Schema validation, null checks      | Great Expectations      |
| UI Tests           | Frontend interactions               | Selenium / Playwright   |

Table 12.1: Testing Strategy

```
1 import pytest
2 import pandas as pd
3 from etl.transform import transform_hotels
4
5 def test_transform_hotels_removes_duplicates():
6     raw = pd.DataFrame({
7         'Place_Name': ['Hilton Cairo', 'Hilton Cairo', 'Marriott Alex'],
8         'Address':    ['Cairo', 'Cairo', 'Alexandria'],
9         'Price':      ['EGP 2500', 'EGP 2500', 'EGP 1800'],
10        'Rating':      ['4.5', '4.5', '4.2'],
11        'Link':        ['url1', 'url1', 'url2'],
12        'Room_Info':   ['King Room', 'King Room', 'Twin Room']
13    })
14    result = transform_hotels(raw)
15    assert len(result) == 2
16    assert 'Price_USD' in result.columns
17    assert result['Rating_Numeric'].dtype == float
```

Listing 12.1: Sample Unit Test — Hotel Transformer

## 13. Project Plan & Milestones

### 13.1 Gantt Timeline (24 Weeks)

| Wk    | Milestone                 | Phase          | Deliverable                             |
|-------|---------------------------|----------------|-----------------------------------------|
| 5     | Team Formation & Planning | Planning       | Project proposal, team roles            |
| 7     | Instructor Review         | Review         | Feedback-incorporated proposal          |
| 9–10  | Requirements Gathering    | Requirements   | Stakeholder analysis, user stories      |
| 11–12 | Data Collection Sprint 1  | Implementation | Static datasets (monuments, museums)    |
| 13–14 | System Design             | Analysis       | ERD, DFD, architecture diagram          |
| 15–16 | ETL Pipeline MVP          | Implementation | Airflow DAGs running, DWH schema live   |
| 17–18 | AI Chatbot MVP            | Implementation | LangChain chatbot answering queries     |
| 19–20 | Recommendation Engine     | Implementation | Budget + interest-based recommendations |
| 21    | API Integration           | Implementation | Google Maps + Weather APIs wired        |
| 22    | Frontend MVP              | Implementation | Streamlit dashboard + chat UI           |
| 23    | Testing & QA              | Testing        | Test report, bug fixes                  |
| 24    | Final Presentation        | Presentation   | Full demo, documentation, video         |

Table 13.1: Project Timeline

### 13.2 Risk Assessment

| Risk                                | Severity | Mitigation                                                |
|-------------------------------------|----------|-----------------------------------------------------------|
| Website scraping blocked (anti-bot) | High     | Rotate proxies, use Selenium stealth, respect rate limits |

| Risk                                | Severity | Mitigation                                            |
|-------------------------------------|----------|-------------------------------------------------------|
| API quota exceeded (Google Maps)    | Medium   | Cache all API responses, implement request throttling |
| LLM API costs overrun               | Medium   | Use open-source alternatives (Ollama/Mistral) for dev |
| Data quality issues in scraped data | Medium   | Great Expectations validation at every pipeline stage |
| Team bandwidth constraints          | Low      | Clear task assignments via GitHub Projects board      |

Table 13.2: Risk Assessment &amp; Mitigation



## 14. Appendices

---

### 14.1 Flask REST API Endpoints

| Method | Endpoint                     | Description                    |
|--------|------------------------------|--------------------------------|
| GET    | /api/places?city=&category=  | Filter places by city/category |
| GET    | /api/monuments               | All 125 monuments              |
| GET    | /api/hotels?budget=&city=    | Hotels filtered by budget      |
| GET    | /api/flights?from=&to=&date= | Latest flight prices           |
| GET    | /api/safety?city=            | City safety score              |
| GET    | /api/weather?city=           | 7-day weather forecast         |
| POST   | /api/chat                    | Send message to Kemet AI       |
| POST   | /api/trip/plan               | Generate full itinerary        |
| GET    | /api/restaurants?city=       | Top restaurants per city       |
| GET    | /api/reviews?place_id=       | Sentiment-analyzed reviews     |

Table 14.1: REST API Reference

### 14.2 Emergency Contacts (Sample Data)

| Service         | Number | Notes                          |
|-----------------|--------|--------------------------------|
| Police          | 122    | National emergency number      |
| Ambulance       | 123    | National ambulance             |
| Fire Department | 180    | National                       |
| Tourist Police  | 126    | Specific for tourist incidents |

Table 14.2: Emergency Numbers (Egypt)

### 14.3 Useful Egyptian Apps for Tourists

- **Talabat** — Food delivery
- **Instashop** — Grocery delivery

- **Vezeeta** — Doctor booking and hospital finder
- **Uber / Careem** — Ride hailing
- **ElGouna Taxi** (Al Gouna specific)
- **Swvl** — Bus booking
- **Egypt Air App** — Domestic flights

# KEMET

*"The Land of the Black Soil"*

---

Bringing Ancient Egypt to the Modern Traveler

Ruwad Masr Al-Raqamia — Data Engineering Track

April 16, 2026